ESPECIFICAÇÃO E IMPLEMENTAÇÃO DE SERVIÇOS

Integrantes:

1. Danilo Henrique de Paulo | 22.222.008-9

2. Jônatas da Silva Gonçalves | 22.123.095-6

3. Kaynã de Deus Ferreira da Silva | 22.123.096-4

4. Pedro Henrique da Fonseca do Nascimento | 22.123.099-8

5. Vinícius do Nascimento Generoso | 22.123.103-8

6. Wallace dos Santos Izidoro | 22.123.104-6

Projeto: All Invest **Data:** 29/10/2026

1. Bounded Context selecionado

Nome do Bounded Context: Desempenho da Carteira

Responsabilidade principal no domínio:

> Apurar e apresentar os resultados dos investimentos, avaliando as posições com base nas cotações de mercado para calcular ganhos, perdas e rentabilidade por ação e da carteira.

2. Capacidades de negócio e serviços candidatos

Liste as capacidades relevantes do Bounded Context. Para cada capacidade, responda às perguntas de análise e indique o(s) serviço(s) candidato(s) proposto(s), se houver.

Capacidade de negócio	Responsabilidade: o que realiza	Consumidores: quem precisa dela?	Por que disponibilizar por serviço?	Serviço(s) candidato(s) e justificativa
Avaliar posições a preço de mercado	Combinar a quantidade de cada ação com sua cotação atual para determinar o valor de mercado da posição.	Report Component.	Centralizar a avaliação permite reutilizar a mesma regra nos cálculos de desempenho.	Serviço de Avaliação de Posições**:** reúne cotações para produzir uma avaliação consistente. Pode ser uma operação interna do Serviço de Desempenho.
Apurar o desempenho por ação	Calcular o valor investido, o ganho ou	Report Component.	Disponibilizar resultados calculados evita	Serviço de Desempenho da Carteira**:** oferece a consulta de desempenho

Capacidade de negócio	Responsabilidade: o que realiza	Consumidores: quem precisa dela?	Por que disponibilizar por serviço?	Serviço(s) candidato(s) e justificativa
	perda e a rentabilidade de cada posição.		que cada consumidor reproduza as regras de desempenho.	por ação, concentrando as regras de cálculo.
Consolidar o desempenho da carteira	Agregar os valores investidos e de mercado das posições e calcular o ganho ou perda total e a rentabilidade consolidada.	Report Component.	Fornecer uma visão consolidada padroniza os resultados apresentados na aplicação e nos relatórios.	Serviço de Desempenho da Carteira**:** oferece a consulta consolidada, reutilizando a avaliação das posições. A rentabilidade total é calculada sobre os valores agregados.

Justificativa da decomposição: explique por que as responsabilidades foram agrupadas em um serviço ou separadas em serviços distintos. Considere coesão, consumidores e fronteiras do domínio.

As responsabilidades foram agrupadas no Serviço de Desempenho da Carteira porque compartilham o objetivo de calcular e apresentar os resultados dos investimentos. Isso evita regras duplicadas para a aplicação e os relatórios. Registro de movimentações, obtenção de cotações e geração de relatórios são mantidos dentro das fronteiras dos seus domínios.

3. Especificação do serviço selecionado

Nome do serviço: Serviço de Desempenho da Carteira.

Capacidade(s) de negócio que disponibiliza: Avaliação das posições a preço de mercado; cálculo de ganhos, perdas e rentabilidade por ação; consolidação do desempenho da carteira;

Responsabilidade do serviço: Apurar e disponibilizar o desempenho dos investimentos, utilizando as posições do investidor e as cotações atuais para calcular os resultados individuais e consolidados.

Consumidores previstos: Report Component, que utiliza os resultados para gerar relatórios, e Frontend, que apresenta o desempenho ao investidor.

Forma de comunicação prevista: Síncrona, por API REST com dados em JSON, para consultar o desempenho. Assíncrona, para acionar o recálculo do desempenho após compras e vendas.

Justificativa da escolha: O serviço reúne regras de cálculo relacionadas e fornece resultados consistentes ao Report Component, evitando duplicação e respeitando as fronteiras do contexto de Desempenho da Carteira.

3.1 Interface e operações

Operação	Finalidade	Entradas principais	Resultado esperado



---

avaliarPosicoes	Avaliar as posições da carteira com as cotações atuais.	Id da carteira.	Lista de posições com ticker, quantidade, preço médio, cotação atual, valor investido e valor de mercado.
consultarDesempenhoAcao	Consultar o resultado de uma ação na carteira.	Identificador da carteira e ticker da ação.	Valor investido, valor de mercado, ganho ou perda e rentabilidade percentual da posição.
obterDesempenhoCarteira	Consultar o desempenho consolidado da carteira.	Identificador da carteira.	Totais investido e de mercado, ganho ou perda total, rentabilidade consolidada e detalhamento por ação.

3.2 Contrato de uma operação

Operação escolhida para detalhamento: avaliarPosicoes

Com a rota: GET /api/performance/portfolios/{portfolio_id}/positions

Elemento	**Especificação**
Parâmetros de entrada	portfolio_id: inteiro positivo obrigatório. O user_id é obtido do usuário autenticado.
Pré-condições	Usuário autenticado; chave de API válida; carteira existente e pertencente ao usuário. Posições disponíveis pelo Investment Component e cotações pelo Market Data Component.
Regras de negócio	Consultar as posições por get_positions(user_id, portfolio_id) e obter a cotação de cada stock_name por get_quote(stock_name)
Resultado / saída	HTTP 200, com uma lista JSON contendo stock_name, company_name, stock_quantity, preco_medio, portfolio_id, portfolio_name, current_price, invested_value e market_value. Carteira sem posições retorna []
Erros ou condições de falha	401: autenticação ou chave de API inválida. 404: carteira inexistente, pertencente a outro usuário ou cotação não encontrada. 422: portfolio_id inválido. 504: tempo de consulta excedido.

---

4. Implementação

Tecnologias utilizadas: Python 3.12, FastAPI, Pydantic, SQLAlchemy, PostgreSQL (Supabase), brapi.dev (cotações), pytest e Swagger UI.

Repositório GitHub: https://github.com/JnSGoncalves/All-Invest (branch feature/relatorios)

Localização dos arquivos relevantes (pasta, módulo ou classe):

- **Serviço:** backend/src/services/performance_service.py (classe PerformanceService).
- **Contrato:** backend/src/interfaces/components.py (IPerformanceService) e backend/src/db/schemas.py (PositionValuationOut, StockPerformanceOut e PortfolioPerformanceOut).
- **API REST:** backend/src/routers/performance.py.
- **Recálculo assíncrono:** backend/src/routers/stocks.py (cadastrar_operacao) e método recalculate do serviço.
- **Injeção de dependências:** backend/src/dependencies/components.py (get_performance_service).
- **Testes:** backend/tests/test_performance_component.py.

Explique brevemente como a implementação realiza as regras definidas no contrato:

A rota valida portfolio_id como inteiro positivo (422) e obtém o user_id do token JWT do usuário autenticado. A chave de API é verificada antes, no middleware da aplicação (401).

O PerformanceService consulta get_positions(user_id, portfolio_id), que recusa com 404 uma carteira inexistente ou de outro usuário, e obtém a cotação de cada ação por get_quote(stock_name). Com esses dados calcula invested_value (quantidade × preço médio) e market_value (quantidade × cotação atual). Carteira sem posições retorna []. Cotação não encontrada resulta em 404 e tempo excedido em 504.

As operações consultarDesempenhoAcao e obterDesempenhoCarteira reutilizam essa avaliação. A primeira acrescenta ganho ou perda e rentabilidade; a segunda calcula a rentabilidade consolidada sobre os totais agregados, e não pela média das ações.

A comunicação assíncrona ocorre depois de cada compra ou venda: POST /api/stocks responde sem esperar e agenda o recálculo em BackgroundTasks, que renova as cotações das posições. Elas ficam 60 segundos em cache, para que consultas em sequência usem o mesmo preço e respeitem o limite da brapi.dev.

O serviço depende apenas das interfaces IInvestmentService, IPortfolioService e IMarketDataService, mantendo as fronteiras do domínio definidas na seção 2. Falhas do provedor de cotações fora do contrato (429, 502 e 503) são repassadas conforme definidas pelo Market Data Component.

A implementação está acessível por meio de uma interface (por exemplo, endpoint/API ou interface equivalente)? **Sim.** O serviço é uma API REST sobre HTTP, com dados em JSON, documentada e testável pelo Swagger UI em /docs. Todas as rotas exigem os cabeçalhos X-API-Key e Authorization: Bearer.

- **avaliarPosicoes:** GET /api/performance/portfolios/{portfolio_id}/positions
- **consultarDesempenhoAcao:** GET /api/performance/portfolios/{portfolio_id}/positions/{stock_name}
- **obterDesempenhoCarteira:** GET /api/performance/portfolios/{portfolio_id}

Os consumidores previstos (Report Component e Frontend) ainda não foram conectados ao serviço. Conforme as orientações do laboratório, a interface gráfica não é necessária e as operações foram testadas pelo Swagger UI.

---

5. Testes e evidências

Registre testes que demonstrem o comportamento esperado e o tratamento de pelo menos uma condição de erro. Os valores abaixo vêm da execução real da API em 01/11/2026, com cotações reais da brapi.dev e dados fictícios em um banco SQLite local. As rotas estão abreviadas e começam em /api/performance/.

Cenário de teste	Dados / condição	Resultado esperado	Resultado obtido / evidência
avaliarPosicoes: sucesso	Carteira com 100 PETR4 a R$ 30,00 e 50 VALE3 a R$ 60,50. GET portfolios/1/positions	HTTP 200 com a lista e os campos do contrato; valor de mercado = quantidade × cotação	HTTP 200. PETR4 a 52,71: investido 3.000,00 e mercado 5.271,00. VALE3 a 62,02: 3.025,00 e 3.101,00. Captura 02
consultarDesempenhoAcao: sucesso	GET portfolios/1/positions/PETR4	HTTP 200 com ganho ou perda e rentabilidade da posição	HTTP 200. Ganho de 2.271,00 e rentabilidade de 75,70%. Captura 03
obterDesempenhoCarteira: sucesso	GET portfolios/1	HTTP 200 com totais e rentabilidade calculada sobre os valores agregados	HTTP 200. Investido 6.025,00, mercado 8.372,00, ganho 2.347,00 e rentabilidade de 38,95%. Captura 04
Carteira sem posições	GET portfolios/2/positions, carteira sem ativos	HTTP 200 com []	HTTP 200 com []. execucao_api.txt
Erro: carteira inexistente ou de outro usuário	GET portfolios/999999/positions e GET portfolios/1/positions com o token de outro usuário	HTTP 404	HTTP 404 nos dois casos: "Carteira não encontrada." Captura 05
Erro: portfolio_id inválido	GET portfolios/0/positions e GET portfolios/abc/positions	HTTP 422	HTTP 422 nos dois casos. Captura 06
Erro: autenticação ou chave de API inválida	Requisições sem X-API-Key, com chave errada, sem token e com token inválido	HTTP 401	HTTP 401 nos quatro casos, por exemplo "API Key não informada". execucao_api.txt
Erro: tempo de consulta excedido	Servidor apontado para um provedor de cotações que aceita a conexão e nunca responde	HTTP 504	HTTP 504 depois de 11 s: "Tempo de resposta da brapi.dev excedido." execucao_api.txt
Erro: cotação não encontrada	Market Data simulado sem cotação para a ação (teste automatizado)	HTTP 404	Aprovado: test_cotacao_nao_encontrada_retorna_404. resultado_testes.txt
Recálculo assíncrono após compra ou venda	POST /api/stocks com uma compra	HTTP 201 sem aguardar o recálculo; recálculo executado em segundo plano	HTTP 201, seguido no log de "Desempenho recalculado em segundo plano: usuário 1, 2 posição(ões), valor de mercado R$ 8372.00". log_servidor.txt

Links ou referências às evidências (testes, capturas de tela, logs ou execução):

Pasta das evidências no repositório: docs/evidencias/lab7/ (branch feature/relatorios).

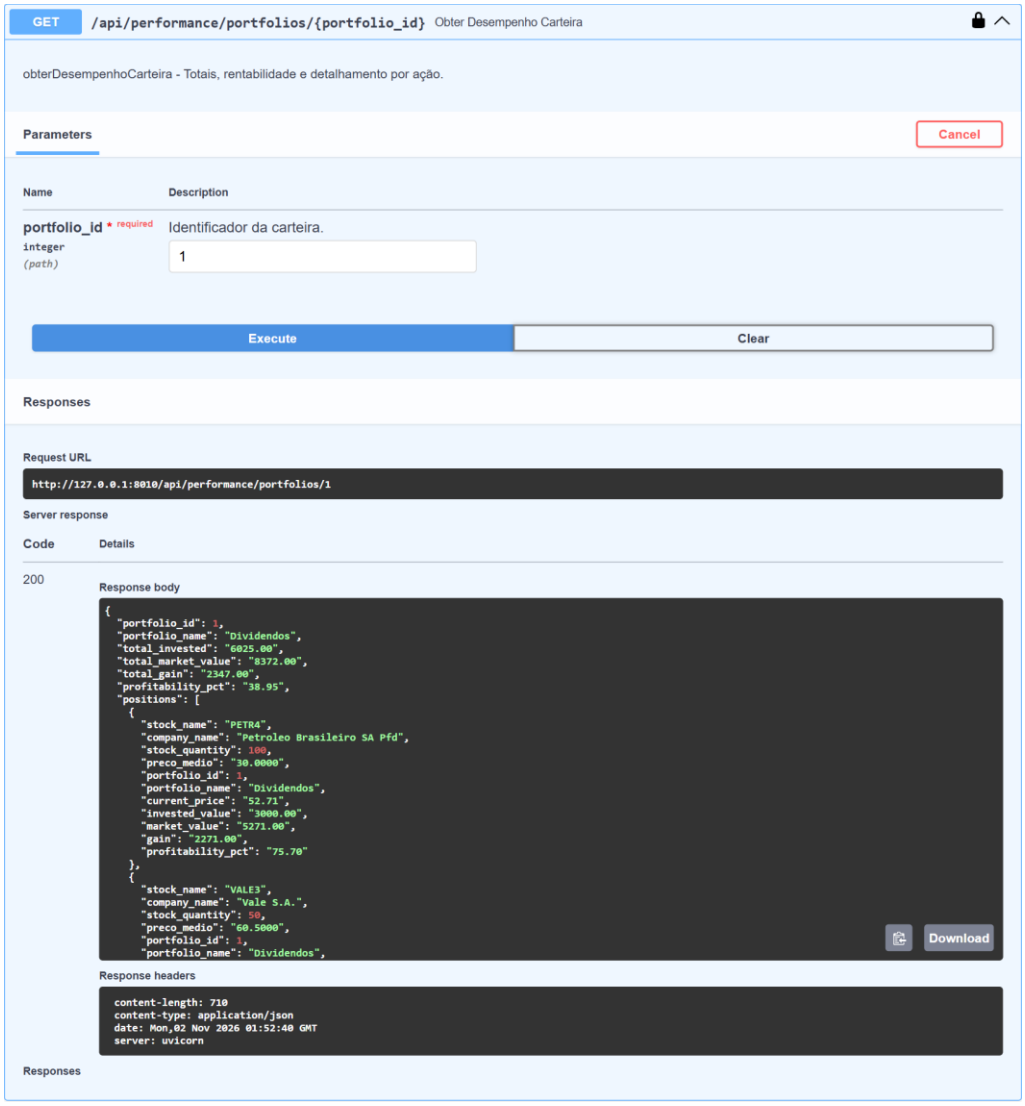
Testes automatizados: resultado_testes.txt, com 129 testes aprovados, 27 deles novos neste laboratório.

Execução real da API: execucao_api.txt (requisições e respostas) e log_servidor.txt (log do uvicorn).

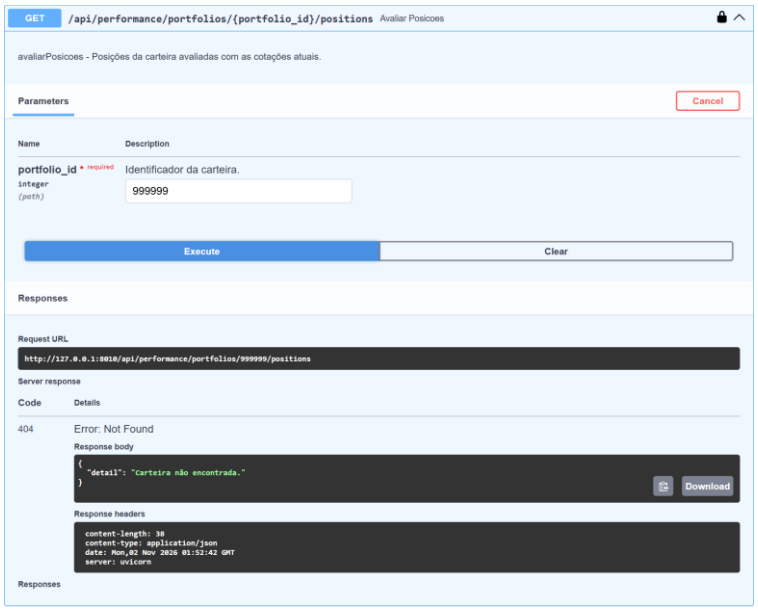
Capturas de tela do Swagger UI: arquivos 01 a 06 em PNG; as capturas 04, 05 e 06 estão reproduzidas no Anexo.

Vídeo de demonstração (opcional): _____

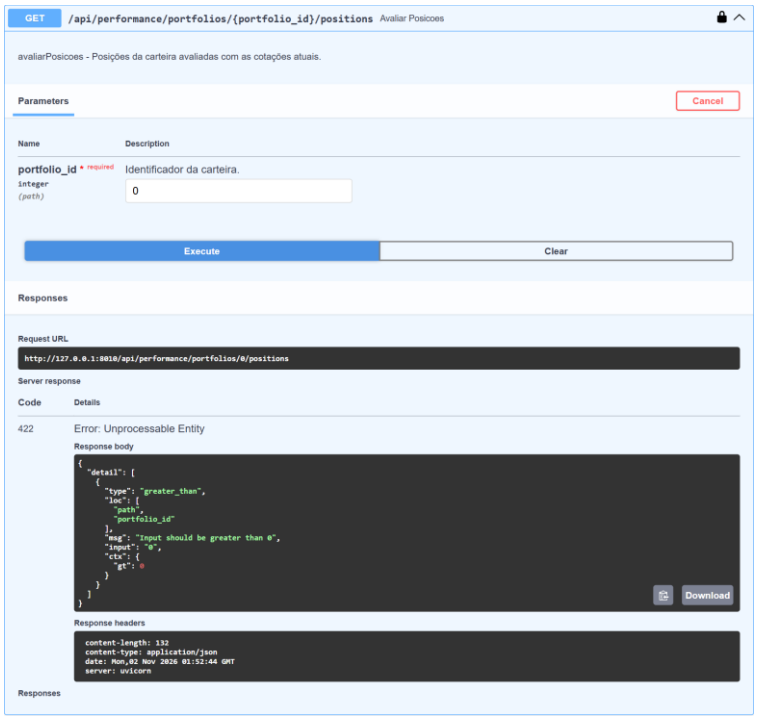
Anexo: capturas do Swagger UI



Captura 04: obterDesempenhoCarteira com HTTP 200.



Captura 05: avaliarPosicoes com carteira inexistente, HTTP 404.



Captura 06: avaliarPosicoes com portfolio_id igual a 0, HTTP 422.